

PAPER_178: CoAnQi 3D Simulation Entity Framework

Abstract

This paper presents a UQFF analysis of astrophysical observables, deriving compressed field equations and observational predictions within the Star-Magic/UQFF framework.

OBJ I/O, Skeletal Animation, and Procedural Landscape

Whitepaper §2.4-J | Thread 381a8fe7 | Session 48

Abstract

CoAnQi exposes a full 3D simulation entity layer bridging UQFF physics computations to visual output. Each MUGESystem maps to a SimulationEntity carrying a 3D mesh, velocity state, and media archive. The framework supports OBJ mesh I/O, real-time skeletal bone animation with SLERP interpolation, procedural terrain generation, multi-viewport rendering, and LaTeX overlay. This paper documents all 3D infrastructure components extracted from ModelLoader.h through CoAnQiNode.py.

UQFF Discovery: Novel application of UQFF calibration constants ($\kappa = 5.0 \times 10^{-4} \text{ day}^{-1}$, $[SSq] = 0.57$) uniquely enabling this analysis — establishing a new connection in the UQFF framework not present in Standard Model treatments.

$$F_U(r, t) = \sum_{i=1}^4 U_{gi} + U_m + U_A - U_{b_i}, \quad \kappa = 5.0 \times 10^{-4} \text{ day}^{-1}, \quad [SSq] = 0.57$$

$$U_{b_i}(r) = \kappa \cdot [SSq] \cdot \frac{GM}{r^2}, \quad \kappa = 5.0 \times 10^{-4} \text{ day}^{-1}, \quad [SSq] = 0.57, \quad \beta_i = 0.61$$

1. Core Data Structures

```
struct MeshData {
    std::vector<glm::vec3> vertices;
    std::vector<glm::vec3> normals;
    std::vector<glm::vec2> texCoords;
    std::vector<unsigned int> indices;
};

struct SimulationEntity {
    double position[3]; // 3D spatial position
    double velocity[3]; // Velocity vector (init from MUGESystem.vexp)
    Object3D model; // Associated mesh
    MediaArchive archive; // Media files captured at simulation time
    void update(double dt) {
        position[i] += velocity[i] * dt; // Euler integration
    }
};
```

2. OBJ Mesh I/O

2.1 loadOBJ

```
MeshData ModelLoader::loadOBJ(const std::string& path) {  
    // Parses: v (vertex), vt (texcoord), vn (normal), f (face)  
    // Face format: f v1/vt1/vn1 v2/vt2/vn2 v3/vt3/vn3  
    // Assigns indices for indexed rendering (deduplicates via posMap)  
}
```

2.2 exportOBJ

```
void ModelLoader::exportOBJ(const MeshData& mesh, const std::string& path) {  
    // Writes: v lines, vn lines, vt lines, f v//vn v//vn v//vn lines  
    // Generates proper 1-indexed OBJ face references  
}
```

3. Texture Loading

```
unsigned int Texture::loadTexture(const std::string& path) {  
    // Uses stb_image (STB single-header library)  
    // Creates OpenGL texture with GL_UNSIGNED_BYTE  
    // Generates mipmaps: glGenerateMipmap(GL_TEXTURE_2D)  
    // Wraps: GL_REPEAT; Filters: GL_LINEAR_MIPMAP_LINEAR  
}
```

4. Shader Pipeline

```
class Shader {  
    unsigned int ID; // OpenGL program ID  
    Shader(const std::string& vsPath, const std::string& fsPath);  
    // Reads GLSL source from file, compiles, links, checks errors  
    void use();  
    void setMat4(const std::string& name, const glm::mat4& mat);  
};
```

5. Camera and Multi-Viewport Rendering

```
class Camera {  
    glm::vec3 position, front, up;  
    float yaw=-90.0f, pitch=0.0f;  
    glm::mat4 getViewMatrix() { return glm::lookAt(position, position+front, up); }  
};  
  
void renderMultiViewports(std::vector<Camera>& cameras, float W, float H) {  
    int n = cameras.size();  
    float vpW = W/n;  
    for (int i=0; i<n; ++i) {  
        glViewport(i*vpW, 0, vpW, H); // divide horizontally  
        // render scene with cameras[i].getViewMatrix()  
    }  
}
```

```

    }
}

```

6. Skeletal Bone Animation

```

struct KeyPosition { glm::vec3 pos; float time; };
struct KeyRotation { glm::quat rot; float time; };
struct KeyScale { glm::vec3 scale; float time; };

struct BoneInfo { std::string name; glm::mat4 offsetMatrix; };

struct Bone {
    std::vector<KeyPosition> positions;
    std::vector<KeyRotation> rotations;
    std::vector<KeyScale> scales;

    glm::mat4 interpolate(float animTime) {
        // Find surrounding keyframes bracket animTime
        glm::vec3 pos = lerp(A.pos, B.pos, factor);
        glm::quat rot = glm::slerp(A.rot, B.rot, factor); // SLERP
        glm::vec3 scale = lerp(A.scale, B.scale, factor);
        return TRS(pos, rot, scale);
    }
};

```

SLERP (Spherical Linear Interpolation) ensures smooth rotational animation without gimbal lock, critical for planetary body spin simulation.

7. Procedural Landscape Generation

```

MeshData generateProceduralLandscape(int width, int height,
                                     float scale, float heightScale) {
    // Height map: h[i][j] = sin(x*scale) * cos(z*scale)
    //               + 0.5*sin(2x*scale) * cos(2z*scale) [octave 2]
    // Normals: computed analytically from gradient or cross-product of edge vectors
    // Indexed triangle mesh: (i,j)?(i+1,j)?(i,j+1), (i+1,j)?(i+1,j+1)?(i,j+1)
}

```

Used to generate terrain meshes for planetary surface simulations (e.g., Earth topography proxy for Ug3/SCm core interaction visualisation).

8. Modeling Stubs

```

void extrudeMesh(MeshData& mesh, float distance); // Extrude along normals
void booleanUnion(MeshData& a, MeshData& b);      // CSG union
// Both are stubs – planned for future plugin implementation

```

9. LaTeX Rendering

```
void renderLaTeX(const std::string& formula, float x, float y) {  
    // Uses MicroTeX library (portable LaTeX renderer)  
    // Renders: F_U, U_g equations, A_u? tensor as overlay on viewport  
}
```

Enables in-simulation display of UQFF equations overlaid on the 3D scene.

10. populate_simulation_entities() — MUGE?Entity Mapping

```
void populate_simulation_entities(  
    std::vector<MUGESystem>& systems,  
    std::vector<SimulationEntity>& entities) {  
    for (auto& sys : systems) {  
        SimulationEntity e;  
        e.position[0] = sys.r;   e.position[1] = 0; e.position[2] = 0;  
        e.velocity[0] = sys.vexp; e.velocity[1] = 0; e.velocity[2] = 0;  
        e.model = load_or_generate_mesh_for(sys.name);  
        e.archive.capture_media(sys.name, timestamp);  
        entities.push_back(e);  
    }  
}
```

11. Python Mirror — CoAnQiNode.py

```
from dataclasses import dataclass  
import OpenGL.GL as gl, vulkan as vk, PyQt5, vtk, pocketsphinx, cv2  
from transformers import pipeline
```

```
@dataclass  
class Object3D:  
    vertices: list[tuple[float,float,float]]  
    normals: list[tuple[float,float,float]]  
    texcoords: list[tuple[float,float]]  
    indices: list[int]
```

```
@dataclass  
class SimulationEntity:  
    position: list[float]  
    velocity: list[float]  
    model: Object3D  
    archive: dict
```

```
class CoAnQiNode:  
    """Python runtime node linking UQFF to Qt5/VTK/Vulkan/OpenCV"""  
    def update(self, dt): ...
```

```
class MainWindow(QMainWindow):  
    """Qt5 main window with embedded VTK widget for CoAnQi"""  
    def __init__(self): ...
```

12. References

- ModelLoader.h/cpp, Animation.h/cpp, Landscape.h/cpp, Modeling.h/cpp, LaTeXRenderer.h/cpp, Camera.h/cpp (thread 381a8fe7)
- CoAnQiNode.py (thread 381a8fe7)
- PAPER_169 (CoAnQi architecture placing 3D in tier context)
- PAPER_177 (FluidSolver that runs inside SimulationEntity context)

Key UQFF calibrated constants: $\kappa = 5.0\text{e-}4 \text{ day}^{-1}$; $[\text{SSq}] = 5.7\text{e-}1$; $H_{\text{SCm}} \approx 9.9\text{e-}1$; $U_{\text{UA}} \approx 1.0\text{e-}4$; $k_{\eta} = 1.0\text{e-}113$; $\beta_i \approx 6.0\text{e-}1$; $G = 6.674\text{e-}11 \text{ N}\cdot\text{m}^2/\text{kg}^2$